

Практическая работа. Введение в RAG и базовый пайплайн.

Курс: Инжиниринг данных.

Цель работы: Понять архитектуру RAG: документы, индекс, retrieval, augmentation, generation. Собрать минимальный прототип вопрос–ответной системы (QA) по набору собственных текстов и сравнить ответы LLM «без retrieval» и с использованием RAG-пайплайна.

Задачи практической работы

1. Познакомиться с архитектурой RAG и ролями основных компонентов: документы, индекс, retrieval, augmentation, generation.
2. Подготовить окружение: Python, библиотеки для LLM и embeddings.
3. Загрузить 5–10 текстовых документов и выполнить простейший chunking.
4. Построить эмбединги чанков и создать простой векторный индекс.
5. Реализовать retrieval релевантных чанков по пользовательскому вопросу.
6. Собрать augmented prompt и получить ответы LLM с использованием контекста.
7. Сравнить ответы модели без контекста и с retrieval, проанализировать ошибки.

1. Подготовка окружения

1.1. Установка необходимых библиотек

Для работы потребуется установленный Python и несколько библиотек:

```
pip install sentence-transformers faiss-cpu openai
```

(при использовании другой LLM/клиента можно заменить openai на соответствующий пакет).

Проверка установки:

```
import sentence_transformers
import faiss
```

1.2. Создание рабочего файла / ноутбука

Создайте:

Jupyter-ноутбук `Practicum_RAG_Baseline.ipynb`

или

Python-файл `practicum_rag_baseline.py`.

В начале файла зафиксируйте:

путь к папке с документами (например, `./docs`),

выбранную embedding-модель (например, `"all-MiniLM-L6-v2"`),

параметры chunking (размер и overlap).

2. Основные концепции RAG

2.1. Что такое RAG

RAG (Retrieval-Augmented Generation) — архитектура, в которой LLM дополняется внешним модулем поиска по собственным данным. Основные элементы:

Документы — исходный корпус текстов (статьи, регламенты, учебные материалы).

Chunking — разбиение документов на небольшие фрагменты (чанки).

Embeddings — векторное представление каждого чанка.

Индекс — структура для быстрого поиска похожих чанков (FAISS, Chroma и др.).

Retrieval — поиск top-k наиболее близких чанков по запросу пользователя.

Augmentation — формирование промпта, куда подставляются найденные фрагменты.

Generation — генерация ответа LLM строго на основе переданного контекста.

2.2. Типовая последовательность шагов

1. Загрузка и предварительная подготовка документов.
2. Chunking текста (фиксированная длина + overlap).
3. Построение эмбеддингов и индекса.
4. Обработка вопроса: encoding вопроса → similarity search по индексу.
5. Сборка промпта: контекст + вопрос + инструкции.
6. Генерация ответа и анализ качества.

3. Построение базового RAG-пайплайна

3.1. Загрузка документов

Задача: Загрузить 5–10 небольших текстовых документов (формат `.txt` или заранее извлечённый текст из PDF/HTML).

Пример шага:

собрать все файлы из папки `./docs`,
сохранить их в список словарей вида `{ "id": ..., "title": ..., "text": ... }`.

Рекомендуется добавить простые метаданные: имя файла, заголовок, при необходимости — раздел/раздел документа.

3.2. Простое разбиение на чанки (chunking)

Задача: Разбить текст каждого документа на фрагменты фиксированной длины с перекрытием.

Рекомендуемый подход:

выбирать размер чанка по числу символов или токенов (например, 500–800 символов),
использовать overlap (например, 100–200 символов), чтобы не «рвать» мысль документу.

Результат: список чанков вида:

```
{  
  "doc_id": ...,  
  "title": ...,  
  "text": "...текст чанка..."  
}
```

4. Embeddings и векторный индекс

4.1. Построение эмбедингов чанков

Задача: Получить векторное представление каждого чанка с помощью выбранной embedding-модели

Шаги:

загрузить модель SentenceTransformer("all-MiniLM-L6-v2") (или аналог),
прогнать через неё список текстов чанков и получить массив эмбедингов.

4.2. Создание простого индекса

Задача: Построить индекс по эмбедингам для быстрого поиска ближайших соседей.

Варианты:

`faiss.IndexFlatL2` — простой индекс по L2-метрике,
или локальное векторное хранилище вроде Chroma.

Результат: объект индекса, в который добавлены все эмбединги чанков.

5. Retrieval и генерация ответа

5.1. Функция retrieval

Задача: По вопросу пользователя найти top-k ближайших чанков.

Шаги:

1. Кодировать вопрос тем же embedding-кодировщиком.
2. Выполнить поиск в индексе (например, `index.search(q_emb, k)`).
3. Вернуть список текстов найденных чанков и при необходимости их метаданные.

5.2. Сборка augmented prompt

Задача: Сформировать промпт, который явно содержит контекст и инструкцию «отвечай только по контексту».

Рекомендуемая структура:

краткая инструкция ассистенту,
блок «Контекст:» со склеенными чанками,
блок «Вопрос: ...»,
явное требование не придумывать ответ при отсутствии информации и, по возможности,
ссылаться на источники.

5.3. Генерация ответа

Задача: Передать построенный промпт в LLM и получить ответ.

В зависимости от выбранного API:

вызвать модель напрямую (OpenAI, локальная модель через Ollama и т.п.),
вывести ответ и, по возможности, список использованных чанков/документов.

6. Эксперименты: «без контекста» против RAG

6.1. Базовый эксперимент

Задача: На одном и том же наборе вопросов сравнить:

1. Ответ модели **без** retrieval (prompt содержит только вопрос).
2. Ответ модели **с** retrieval (prompt = контекст + вопрос).

Рекомендуется подготовить 3–5 вопросов, чьи ответы явно содержатся в документах.

6.2. Анализ ошибок

При сравнении обращать внимание на:

случаи, когда модель без контекста «галлюцинирует» или выдаёт общее знание, не связанное с вашими документами;
случаи, когда RAG-подход даёт более точный, цитируемый ответ;
ситуации, когда retrieval подобрал нерелевантные чанки и это испортило ответ.

7. Задания для самостоятельной работы

Задание 7.1. Подготовить корпус из минимум 5 текстовых документов по одной тематике (например, выдержки из учебника, методички, регламентов).

Задание 7.2. Реализовать функции:

загрузки документов;
простого chunking с overlap;
построения эмбедингов и индекса;

retrieval top-k чанков;

генерации ответа по augmented prompt.

Задание 7.3. Сформировать таблицу экспериментов:

вопросы;

ответ без retrieval;

ответ с retrieval;

комментарий (где и почему RAG сработал лучше/хуже).

Задание 7.4. Сформулировать краткие выводы (3–5 предложений) о сильных и слабых сторонах базового RAG-прототипа.

8. Требования к отчёту

Отчёт по работе должен содержать:

1. Титульный лист

- название курса и практической работы;
- ФИО студента, группа;
- дата выполнения.

2. Введение

- цель работы;
- краткое описание архитектуры RAG и её назначения.

3. Построение базового пайплайна

- схема шагов: документы → chunking → embeddings → индекс → retrieval → augmentation → generation;
- краткое описание каждой стадии.

4. Реализация прототипа

- фрагменты кода (загрузка документов, chunking, embeddings, индекс, retrieval, prompt);
- пример одного–двух запросов и ответов модели.

5. Эксперименты и анализ

- таблица с вопросами и ответами «без RAG» и «с RAG»;
- разбор ошибок: «ответ без контекста» против «ответ с retrieval».

6. Выводы

- чему студент научился в ходе работы;
- где такой RAG-подход может быть полезен;
- какие слабые места выявлены у базового прототипа.